

Lia — What We Built

A record of the project: architecture, measurements, fixes, and honest limits.

Lia is a companion AI that runs entirely on one laptop. No cloud APIs, no accounts, no data leaving the machine. This document records what she is made of, what changed while building her, and — just as importantly — what is measured rather than assumed.

The constraint that shaped everything. A 4 GB graphics card. It decided the model size, the context window, where speech recognition runs, and why several obvious-looking improvements were rejected.

What she is

Piece	Choice	Why
Brain	Ollama + llama3.2:3b	Fits in 4 GB VRAM
Memory	SQLite + nomic-embed-text	Facts, past turns, documents
Voice out	Piper (en_US-amy-medium)	Local neural speech, swappable
Voice in	faster-whisper small.en + Silero VAD	CPU, so it never competes for VRAM
Interface	System tray app, no window	Always there, never in the way

The files

File	Role
config.py	Every prompt and tunable. The first file to edit.
db.py	SQLite schema: facts, memories, diary, documents
llm.py	Ollama wrapper — chat, streaming, embeddings, load/unload
memory.py	Embedding, retrieval, fact extraction and sanitising
diary.py	End-of-session reflection, and the startup greeting
library.py	Reads PDFs and notes you hand her
internet.py	The one deliberate offline exception — weather, online lookups
media.py	Windows media control (SMTC) — play/pause/skip/now playing
alarms.py	Spoken alarms and timers — regex-parsed, never guessed by the model
voice.py	Speech out, speech in, voice detection, wake word
main.py	The conversation loop and session lifecycle
app.py	Tray app wrapper for running her in the background

The layering is strict: **config** knows nothing; **db** and **llm** are the only modules that touch the outside world; **memory**, **diary** and **library** hold the judgement calls; **voice** and **main** are the interface. A future GUI can reuse everything below **main** untouched.

What was measured

Every number here came from running the thing, not from estimating it. Several contradicted what seemed obvious.

Context window — the biggest single find

Ollama defaults llama3.2 to a 32k context. On a 4 GB card that inflates the model past what fits, and it silently runs most of it on the CPU. Nothing errors. It just feels slow.

num_ctx	Model size	On GPU	Speed	Full reply
32768 (default)	6.4 GB	37%	14.5 tok/s	6.28s
8192 (chosen)	3.4 GB	68%	16.9 tok/s	4.72s
4096	2.9 GB	80%	17.4 tok/s	4.13s

8192 was chosen over 4096 to leave room for the system prompt, remembered facts, retrieved memories and eight turns of history without silently dropping the oldest ones.

Latency, start to first sound

Stage	Time
Embedding your words for recall	2.1s
Ollama to first token (warm)	2.4s
Piper to first audio	~1s
Total, warm	~5.5s
Cold start, models unloaded	9.4s to first token

Speech recognition size

base.en heard “My sister Ania” where small.en heard “My sister Anaya”. Names matter for a companion, and the extra 1.3s is nothing against a multi-second reply, so small.en won.

Voice activity detection

speech chunks: max prob 1.00, 47/55 over threshold

silence chunks: max prob 0.00, 0/31 over threshold

Document indexing

A 37-page PDF became 91 passages in about four minutes — roughly 2s per passage, all of it embedding calls. Once per file, unless the file changes.

Bugs found and fixed

Grouped by what they broke. Most were silent — nothing crashed, the behaviour was just quietly wrong, which is the harder kind to notice.

She looked frozen

- **No streaming.** Replies were fetched whole before printing, so 20–60s passed with a blank terminal. Now tokens stream, with ...remembering / ...thinking / ...saving markers over the silent stretches.
- **Unicode crash.** llama3.2 emits em-dashes constantly; writing one to a non-UTF-8 Windows console raised mid-reply and killed the session. stdout is now pinned to UTF-8.

Her memory was quietly corrupting itself

- **Junk facts overwrote good ones.** Fact extraction stored name: "you" and favourite_food: "None", silently replacing real values. A deterministic sanitiser now rejects pronouns, placeholders, nested objects and over-long values.
- **Half her recall was her own voice.** Retrieval searched every stored turn including her own replies, so two of four recall slots returned things *she* had said — unlabelled, which she then read back as fact. This produced an invented emotional memory. Retrieval is now limited to your turns.
- **No sense of time.** Timestamps were stored and never used. Memories now arrive as “*yesterday, they said...*” instead of floating in an eternal present.
- **Renaming didn't stick.** A name change reverted because end-of-session extraction re-derived the old one from stored turns. Protected facts can no longer be overwritten by extraction.

The diary was writing fiction

- **Role confusion.** Transcripts were labelled user: / assistant:, and a 3B model doesn't reliably map *assistant* to itself — so it narrated entries as though it were the human. Turns are now labelled **Them** and **Lia**.
- **Invented a childhood.** She wrote about her grandfather bringing dumplings to family gatherings. The diary prompt now states plainly that she has no past outside these conversations.
- **Nobody ever read it.** Entries were written and never loaded again. Her startup greeting is now generated from her recent diary, which is what makes it feel like continuity rather than a canned line.

Voice-specific

- **Stage directions read aloud.** She emitted **soft, gentle tone** and Piper read the asterisks. Speech is now stripped of stage directions and markdown; the text output keeps them.
- **She greeted the dog.** In a flat list of facts, pet_name: Biscuit reads exactly like name: Warlock. The person is now named separately and explicitly.
- **Adding voices changed her voice.** VOICE_NAME = "auto" picked the first file alphabetically, so downloading a new one silently replaced her. Now pinned by name.

The packaged app

- **Frozen paths.** Inside a bundle __file__ points at a temp folder deleted on exit. Her database, voices and log now anchor to the executable's own directory.

- **She locked her own files.** The app launched `ollama serve` as a child, which inherited her folder as its working directory — Ollama's workers then held handles inside it and blocked every rebuild. It now starts from a neutral directory.
- **Half-written builds.** Rebuilding wiped the folder Windows autostart pointed at, producing *“Failed to import embedded python interpreter”*. Builds now go to a staging folder and swap in only when complete, so a failed build leaves the working app untouched.
- **Re-reading everything on every start.** Document paths were absolute, so the same PDF under `dist\Lia\library` looked like a different file and was re-embedded from scratch — four minutes before she would say a word. Paths are now relative to the library folder.

What was added

Voice, both directions

Piper for speech, faster-whisper for listening, Silero VAD for knowing when you start and stop. Replies are chunked by sentence and spoken as they generate, so she starts talking before the full reply exists — essential when generation takes seconds.

Your own voices drop into `voices/` as a matched `.onnx + .onnx.json` pair. Any engine exposing `.say(text)` and `.name` can be added alongside Piper.

Wake word, without a wake-word model

“Lia” isn’t in any pretrained wake-word model, and training one needs a dataset. Since Whisper already transcribes everything, the name is matched **on the transcript** instead — no new dependency, no training, and it tolerates the spellings Whisper actually produces (Leah, Lea, Liya).

After she replies you get 60 seconds of free conversation; after her startup greeting, 30. The shorter greeting window exists because she greets the room at login whether or not anyone is there.

Interrupting her, on speakers

The mic stays open while she speaks. Since **she knows what she is currently saying**, anything heard is compared against her own words — her voice returning through the speakers is recognised and ignored, yours cuts her off. Clean on headphones; on speakers it depends on how clearly the mic picks her up.

```
heard 'your grandmother Kamala taught you to cook' -> her own voice, ignored
heard 'wait stop'                                   -> you, cuts her off
```

Spoken instructions

A small phrase list is matched before anything reaches the model, so “Lia, stop listening” switches the mic off instead of becoming something she muses about.

A library, on your terms

A single folder you hand her documents through. She never crawls your drive and never reads anything you haven’t asked for — a file sitting there is available, not absorbed. Say “Lia, read my files” and she indexes it, then cites the document and page when answering.

Document passages are kept separate from her memories and labelled as things she read, not things you told her. Blur those and she starts attributing a document’s opinions to you.

Running as an app

A tray icon with no window, autostart at login, and a packaged executable that needs no Python installed. This only became possible once voice detection removed the need for a keyboard — a windowless app can’t read one.

She waits for Ollama on startup and launches it if needed, because at login she wins that race nearly every time.

Memory that follows the conversation

The keep-alive timer resets on every request, so its value never affects an active conversation — it only decides how long ~3.7 GB of VRAM sits idle afterwards. She now releases the models explicitly when a conversation closes, and starts reloading the moment you begin speaking, so the load overlaps with transcription.

	Fixed 30m timer	Now
During a conversation	loaded	loaded
5 minutes after	held	held
15 minutes after	held	released
Returning cold	2.9s	6.9s

A real latency bug, found by measuring rather than guessing

Asked to look into reply latency, the actual per-turn cost was measured rather than assumed: silence detection (0.9s), Whisper transcription (~1.9s), then a genuine bug — **the same sentence was being embedded twice**, once for memory recall and once for library search, at ~2.2s each. Fixed by sharing one embedding call between both, and by skipping retrieval entirely for short replies ("yeah", "okay") where a 2-second lookup never had anything to return anyway.

Separately, she was addressing the person by name in nearly every reply because the system prompt said “Address them as {name}” and she took it literally. Softened to prefer “you”, the name held back for the start of a conversation or something that matters.

The one deliberate exception to offline

Everything up to this point works with zero internet connection, by design. Two narrow, explicit, optional additions:

- **Weather** — a live source (Open-Meteo), geolocated from IP, no API key needed at all. Answered directly from data rather than asked of any LLM, which would only guess.
- **“Look that up”** — a factual question, answered online only when explicitly asked. Two providers: **Groq** is fast but is just a bigger version of the same kind of model running locally — a plain chat-completions call to it has no more access to today's news than the local model does either. **OpenRouter**, with its :online mode, runs an actual web search first — verified with a real, current, cited answer (a 2025 sporting result) that neither the local model nor a plain Groq call could have known from training data. Preferred automatically when both are configured.

Path	Measured latency	Note
Weather	~1.2s	no LLM involved
Connectivity check	~0.3s	run before anything else
Groq lookup	0.4–0.8s	fast, but guesses
OpenRouter (:online)	2.4–4.0s	slower, because it actually searches

Trigger-phrase detection for both runs on every ordinary turn, measured at 0.002ms per turn — confirmed to add no perceptible cost to conversation that never touches either feature.

Windows media control

Built against System Media Transport Controls — the same system behind the taskbar's media flyout and hardware media keys — rather than integrating one specific player. This means it works with whatever is actually playing (Spotify, a browser tab, Windows Media Player) without Lia needing to know which. Verified against a real, live session already running on the machine.

A second voice engine: tried, then removed

A set of voice model files (KittenTTS, CPU-only, 25–80MB, its own voice set including “Bella”) was investigated, got working end to end after fixing three separate upstream packaging bugs (a dependency version that doesn't exist on PyPI, a hardcoded Windows path to a system eSpeak install, a data path baked in from the CI machine that built it) — and was then removed entirely by request. The active voice stayed on Piper's “amy” throughout; it was never switched without being asked to.

What actually living with her surfaced

All of this came from real use after the build, not from further testing in isolation — the kind of thing that only shows up once someone is actually relying on her day to day.

The CPU discovery

Reported as “over 50% CPU, competing with other work.” Measured directly on the running process before touching anything:

CPU (one core = 100%)	
Idle, before the fix	~290%, sustained, sitting completely idle
Idle, after the fix	~4%, measured on the real packaged app

Root cause: the voice-activity-detection library's ONNX runtime defaults to a **spinning thread pool** — burning CPU continuously waiting for the next audio chunk instead of sleeping between them, regardless of how little real work arrives. A single environment variable, `OMP_WAIT_POLICY=PASSIVE`, set before anything else loads, was the entire fix — confirmed with a synthetic tight-loop benchmark first (389% → 94%), then against the actual project code path, then against the real running app. No measurable effect on how quickly she notices speech.

Alarms and timers, built from nothing

“I told her to keep an alarm, she couldn't” — because the feature never existed. Built as its own module, deliberately **not** routed through the local model: a timer has to fire at the right second, and asking a 3B model to extract “20 minutes” reliably is exactly the kind of precise, literal task it fails at (see the Morse code experiment). Regex-based parsing instead, gated on an explicit keyword (“alarm”, “remind”, “timer”, “wake me”) as well as a time expression, so ordinary conversation like “I usually get up at 7” can't silently schedule a bogus alarm.

A real bug was caught before it shipped: “set a timer for 10 minutes” was initially misread as the clock time 10:00, because the absolute-time pattern matched “for 10” before the relative-duration pattern got a chance. Fixed by making sure a duration expression is always claimed first.

Alarms persist in the database across restarts, and a background watchdog checks for due ones every 10 seconds regardless of what else is happening — verified live, including forcing a near-term alarm to confirm it actually interrupts and speaks.

Music: honest failure instead of a fabricated answer

“Play music” didn't work either, for a different reason: Windows media control is a *remote* for something already playing elsewhere (Spotify, a browser tab) — it cannot conjure a track into existence. Worse, when nothing was playing, she was caught **inventing a fake “jazz playlist”** rather than saying she had nothing to check. Fixed at the source: media commands now speak their result, and failure produces an honest line (“there's nothing loaded anywhere for me to play”) instead of silence or a guess. Command phrasing was also broadened, and a real inconsistency was found and closed along the way — typed input never went through the same natural-language command matching that spoken input did.

Running twice

Raised by a plain question — “do I need to restart, or can I just run her file?” — which exposed a real gap: nothing stopped a second copy starting while the autostart one was still up, which would mean two processes fighting over the same microphone and speakers. A Windows mutex now blocks a second instance outright, tested across genuinely separate processes and confirmed to release cleanly even after a forced kill, not just a clean exit.

A close call: two leaked keys

While preparing to publish the repository, both live API keys were found sitting in git history — not in project source, but in Claude Code's own permission-cache files (`.claude/settings.json`, `settings.local.json`), which had recorded the literal command strings used to set them as environment variables. Confirmed the commit had genuinely been pushed (local main and origin/main matched exactly).

A .gitignore alone would not have fixed this: git ignores that file for anything already tracked, and it does nothing to a secret already sitting in a pushed commit's history. Both keys were rotated — the only real fix once a secret has been public, however briefly — and 2,019 files (the settings folder, build output, databases, logs) were untracked from git without touching a single file on disk, so the same leak can't repeat.

What she still can't do

Recorded honestly, because a companion that overstates itself is worse than one with known gaps.

Precise symbolic tasks — and teaching her doesn't help

Asked for Morse code from her own knowledge, she gave `S = ". -"` (wrong) and `CAT = ". -"` (wrong). Given a **perfect reference table in context** — taught exactly as you'd teach a person — she still produced `CAT = ". - . - -"`.

She fails while looking at the answer. This is not a memory problem, so no amount of teaching fixes it. A 3B model doesn't resolve individual characters well enough for character-by-character lookup. The right fix for tasks like this is code she can call, not knowledge she is given.

She still embroiders

Speaker-filtered retrieval removed the largest source of invented memories, but not all of it. She has produced lines like “*I was starting to think you'd forgotten about your favourite spot here*” about things that never happened. Better, not solved.

Open items

- **No way to forget.** Every turn is stored in plain text with no expiry and no /forget. This matters most now that she starts at login, and now that an optional path exists for a question to leave the machine at all.
- **Duplicate facts.** Keys drift — `sister_name` and `visiting_sister_name` both existed for the same person. Nothing merges them.
- **Wake word still transcribes everything.** She only *responds* to her name, but she processes all audible speech to check for it. A true wake word would gate before transcription.
- **No tests.** Notable because every real bug so far has been silent — nothing crashed, the data just quietly got worse.

- **Whole-document questions.** She retrieves passages, not books. “Summarise this 300-page PDF” will not work; “what does it say about X” will.
- **Music playback control is Windows-only.** Built against System Media Transport Controls, which has no equivalent on other platforms, and it can only control something already playing — it cannot launch or choose music from nothing.

Resolved since the last pass: the packaged app's stray ~77MB of unused spacy dependency is gone (it came in with KittenTTS, now fully removed); a second process could no longer start alongside a running one.

Scanned PDFs

There is no OCR. If text isn't selectable in a PDF viewer, there is nothing to extract.

The dials

All in `config.py`. These are the ones worth knowing about.

Setting	Default	Change it when
NUM_CTX	8192	Never, without checking <code>ollama.ps</code> for the CPU/GPU split
OLLAMA_KEEP_ALIVE	30m	Now only a backstop — release happens on idle
RELEASE_MODELS_WHEN_IDLE	True	You'd rather hold VRAM than reload
PREWARM_ON_SPEECH	True	You want no speculative loading at all
IDLE_MINUTES	12	Conversations should close sooner or later
WAKE_WORD_ENABLED	True	You want her to answer everything she hears
CONVERSATION_WINDOW_SECONDS	60	You repeat her name too often, or too rarely
GREETING_WINDOW_SECONDS	30	She answers noise after login (lower it)
VAD_SILENCE_SECONDS	0.9	She cuts you off mid-thought (raise it)
VAD_THRESHOLD	0.55	Background noise triggers her (raise it)
BARGE_IN	True	She interrupts herself on speakers
ECHO_MATCH_RATIO	0.5	She ignores you while talking (raise it)
WHISPER_MODEL	small.en	Names come out wrong (go bigger)
VOICE_NAME	en_US-amy-medium	You want a different voice
VOICE_ENGINE	piper	"system" forces the built-in Windows voice
LIBRARY_AUTO_READ	False	You want documents read without asking
RETRIEVAL_MIN_WORDS	3	Short replies still feel slow (raise it), or memory feels thin (lower it)
OPENROUTER_ONLINE	True	Live search on "look that up" instead of a guess

Both `GROQ_API_KEY` and `OPENROUTER_API_KEY` are read from environment variables, never from this file — set with `[Environment]::SetEnvironmentVariable(...)`, not by editing `config.py`.

Rebuilding the app

```
powershell -ExecutionPolicy Bypass -File LIA\build_exe.ps1
powershell -ExecutionPolicy Bypass -File LIA\autostart.ps1 install -Exe
powershell -ExecutionPolicy Bypass -File LIA\autostart.ps1 remove
```

The build stages into a temporary folder and swaps at the end, so a failed build never leaves you without a working app.